
Airport Management System

JavaFX Edition - Complete Educational Guide
Interview Preparation | University Defense | Exam Study

Technologies: Java 17 | JavaFX 11 | OOP | Socket Programming | Multithreading | Collections |
Exception Handling | Python TCP Server

This is a Java project. The only Python file is server.py (chat server).

27 Java Source Files | 1 Python Script | Complete GUI | LAN Chat

Table of Contents

PART I - FOUNDATION

1. Project Overview & Purpose
2. Architecture & Folder Structure
3. Program Startup & Execution Flow

PART II - CLASS-BY-CLASS ANALYSIS (All 27 Java Files)

4. Launcher.java - The Entry Point
5. Main.java - JavaFX Application Core
6. Colors.java - Color System
7. Person.java - Abstract Base Class
8. Employee.java - Abstract Employee
9. Fighter.java - Pilot Class
10. modiefoun.java - Co-Pilot Class
11. Passengere.java - Passenger Class
12. status.java & Flight.java - Flight Model
13. checking.java - Input Validation
14. adding.java - Data Storage (Static Lists)
15. Sidebar.java - Navigation Panel
16. TopBar.java - Search & Status Bar
17. WelcomePage.java - Splash Screen
18. DashboardPage.java - Overview Dashboard
19. FlightsPage.java - Flight Management
20. PassengersPage.java - Passenger Management
21. PilotsPage.java - Pilot Management
22. CoPilotsPage.java - Co-Pilot Management
23. ReportsPage.java - Report Generation
24. ChatPage.java - Chat GUI
25. SettingsPage.java - System Info
26. chatclient.java - Socket Chat Client
27. Chatwindowcontroller.java - Legacy (Unused)
28. App.java - Legacy Original GUI (Unused)
29. normalemployee.java - Legacy (Unused)
30. server.py - Python TCP Chat Server

PART III - TECHNICAL DEEP ANALYSIS

- 31. OOP Principles in This Project
- 32. JavaFX GUI Analysis
- 33. Socket Programming & Networking
- 34. Multithreading
- 35. File Handling
- 36. Exception Handling
- 37. Collections & Data Structures
- 38. Design Patterns

PART IV - EXAM & INTERVIEW PREPARATION

- 39. Execution Flow Timeline
- 40. Memory & JVM Analysis
- 41. Interview Questions (Per Class + Answers)
- 42. 100+ University Defense Questions
- 43. Suggested Improvements
- 44. ASCII Diagrams
- 45. Complete Study Guide & Cheat Sheet
- 46. Glossary of Java Keywords
- 47. 50+ Question Quiz with Answers

PART I: FOUNDATION

1. Project Overview & Purpose

What is this project?

This is a desktop application for managing an airport. It lets you manage flights, passengers, pilots, and co-pilots. It also has a LAN chat feature and report generation.

Why does it exist?

It was built as a university project to demonstrate Object-Oriented Programming, JavaFX GUI development, socket networking, and collections management.

What would happen without it?

Airport staff would need to manage everything on paper or with multiple separate tools. This system brings everything into one unified, graphical interface.

Key Features:

- **Dashboard** - Overview with stats, management grid, flight table, quick actions
- **Flight Management** - CRUD with inline status change (SCHEDULED/BOARDING/DEPARTED/CANCELLED)
- **Passenger Management** - Add, view, search, delete passengers
- **Pilot Roster** - Salary, flight hours, license management
- **Co-Pilot Roster** - Salary, hours, language management
- **Reports** - 5 report types with save-to-file (.txt)
- **LAN Chat** - Real-time socket chat via Python server
- **Search** - Global search bar filters current page's table
- **Welcome Screen** - Animated landing page with airplane pulse

***Important Architecture Note:** All 27 Java files use JavaFX (not Swing). The GUI is built entirely in Java code - no FXML, no Scene Builder. The only Python file is server.py for the chat server.*

2. Architecture & Folder Structure

The project is organized into a clear folder structure. The src/ folder contains all Java source files. There are no Java packages - all classes are in the default package. The lib/ folder contains JavaFX JAR copies. The server/ folder has the Python chat server.

```
project/
├── .vscode/
│   ├── launch.json # VS Code launch config
│   └── settings.json # JavaFX lib references
├── bin/ # Compiled .class files
├── lib/ # JavaFX JAR copies
├── server/
│   └── server.py # Python TCP chat server (port 5555)
├── src/
└── Launcher.java # Entry point (bypasses JDK 17 module check)
```

```

■■■■ Main.java # JavaFX Application: root layout, navigation
■■■■ Sidebar.java # Left sidebar: logo, nav buttons
■■■■ TopBar.java # Top bar: title, search, date/time
■■■■ Colors.java # All color constants
■■■■ WelcomePage.java # Splash/welcome screen
■■■■ DashboardPage.java # Dashboard with stats & tables
■■■■ FlightsPage.java # Flight management
■■■■ PassengersPage.java # Passenger management
■■■■ PilotsPage.java # Pilot management
■■■■ CoPilotsPage.java # Co-pilot management
■■■■ ReportsPage.java # Report generation
■■■■ ChatPage.java # Chat GUI
■■■■ SettingsPage.java # System info
■■■■ Flight.java # Flight class
■■■■ Flighter.java # Pilot class
■■■■ modiefoun.java # Co-pilot class
■■■■ Passengere.java # Passenger class
■■■■ Person.java # Abstract base class
■■■■ Employee.java # Abstract employee class
■■■■ status.java # FlightStatus enum
■■■■ adding.java # Static data storage
■■■■ checking.java # Input validation
■■■■ chatclient.java # Socket chat client
■■■■ App.java # Legacy (unused)
■■■■ normalemployee.java # Legacy (unused)
■■■■ Chatwindowcontroller.java # Legacy (unused)

```

Class Categories:

Category	Files	Purpose
Entry & Core	Launcher, Main	App startup, navigation, root layout
Data Storage	adding, checking	Static ArrayLists, input validation
Layout	Sidebar, TopBar, WelcomePage, Colors	Navigation, search bar, welcome screen, color system
Legacy	App, normalemployee, Chatwindowcontroller	Unused files from earlier development

3. Program Startup & Execution Flow

How does the program start?

You cannot run `Main.java` directly because JDK 17 has a module system that blocks launching JavaFX apps directly. The solution is `Launcher.java` - a simple class that calls `Main.main()`. This bypasses the module check.

Step-by-Step Startup:

- 1. VS Code runs `Launcher.main()` (configured in `launch.json` with `--module-path` for JavaFX)
- 2. `Launcher.main()` calls `Main.main(args)`
- 3. `Main.main()` calls `launch(args)` - the JavaFX `Application.start()` method
- 4. `start(Stage primaryStage)` is called by the JavaFX runtime
- 5. A `StackPane (baseRoot)` is created as the root of the scene
- 6. A `BorderPane (root)` is created and added behind the welcome screen
- 7. `Sidebar` is created and set as `root.setLeft(sidebar)`
- 8. `TopBar` is created and set as `root.setTop(topBar)`
- 9. A `StackPane (centerStack)` is created and set as `root.setCenter(centerStack)`
- 10. `navigateTo('dashboard')` is called - `DashboardPage` is added to `centerStack`
- 11. `WelcomePage` is created and added ON TOP of everything in `baseRoot`
- 12. Scene is created (1600x900), `primaryStage.show()` is called
- 13. User sees the welcome screen (dark gradient, airplane, 'Enter System' button)
- 14. User clicks 'Enter System' -> welcome overlay is removed from `baseRoot`
- 15. User now sees the dashboard with sidebar, topbar, and center content

Why Launcher.java exists:

JDK 17 introduced strong encapsulation. When you try to run `Main.main()` directly, the JVM checks if `Main` extends `Application` and refuses to launch it because JavaFX needs to be on the module path. `Launcher.java` is a separate class that does NOT extend `Application`, so it can be the entry point. It then calls `Main.main()` which launches the JavaFX app. This is a common workaround.

launch.json configuration:

```
vmArgs: --module-path /usr/share/openjfx/lib --add-modules
javafx.controls,javafx.fxml,javafx.graphics,javafx.base
```

PART II: CLASS-BY-CLASS ANALYSIS

This section explains every Java file in the project. Each file is analyzed for purpose, fields, methods, OOP concepts, and interview relevance.

4. Launcher.java - The Entry Point

Purpose: Bypass JDK 17 module check to start the JavaFX application.

Why it exists: JDK 17 prevents launching a class that extends Application from the module path. Launcher is a plain class that calls Main.main(args), acting as a bridge.

What happens without it: The app would crash with a module system error when trying to launch Main directly.

Code Analysis:

```
public class Launcher {
    public static void main(String[] args) {
        Main.main(args);
    }
}
```

Line by line:

- **public class Launcher** - Public class, no package, no extends/implements
- **public static void main(String[] args)** - Standard JVM entry point
- **Main.main(args)** - Delegates to Main which calls launch(args) for JavaFX
- Memory: No objects created. Just a method call chain.

OOP Concepts: None - this is a utility entry point.

Interview Q: Why do you need a Launcher class instead of running Main directly? Answer: JDK 17 module system prevents launching classes that extend Application from the module path. Launcher bypasses this.

5. Main.java - JavaFX Application Core

Purpose: Creates the root layout (BorderPane), manages page navigation, handles search routing, and shows the welcome screen.

Field	Type	Purpose
root	BorderPane	The main layout: sidebar left, topbar top, content center
topBar	TopBar	Reference to update title when navigating

Field	Type	Purpose
baseRoot	StackPane	The true root of the scene. Holds root (main app) + welcome overlay

Key Methods:

Method	Purpose	Key Detail
start(Stage)	JavaFX entry point. Creates entire UI.	Creates baseRoot (StackPane) > root (BorderPane) > sidebar + topBar + centerStack
filterSearch(String)	Routes search to current page	Checks type of current node and calls filterTable() on it

The Welcome Screen Pattern (Critical):

Main uses a StackPane as the base root. The main app (BorderPane) is added first (behind). The WelcomePage is added second (on top). When the user clicks "Enter System", the welcome overlay is removed, revealing the main app underneath.

Why WelcomePage[] welcomeHolder pattern? Java requires local variables to be initialized before use in lambdas. Since welcomeHolder[0] is assigned AFTER the WelcomePage constructor runs (but the constructor needs the lambda), we use a holder array: the lambda captures the array reference (final), and reads welcomeHolder[0] later when the button is clicked - by then it's been assigned.

***Error Handling:** navigateTo() is wrapped in try-catch. If any page creation fails, an error card is shown instead of a blank screen.*

6. Colors.java - The Color System

Purpose: Centralized color constants for the entire UI theme.

Why it exists: Instead of hardcoding hex values like "#2F3A1D" scattered across 20+ files, all colors are defined once in Colors.java. If you want to change the sidebar color, you change ONE line and it updates everywhere.

What happens without it: You'd have hundreds of hardcoded color strings across files, making theme changes a nightmare.

Color Families: Each page type has **LIGHT (bg)**, **MID (border)**, **SOLID (accent)** variants.

Family	LIGHT	MID	SOLID	Page
Flights	#E3F2FD	#90CAF9	#1976D2	Blue

Family	LIGHT	MID	SOLID	Page
Pilots	#FFF3E0	#FFCC80	#E65100	Orange
Reports	#E0F7FA	#80DEEA	#00838F	Teal
Settings	#F5F5F5	#E0E0E0	#424242	Grey

7. Person.java - Abstract Base Class

Purpose: The root of the class hierarchy. Defines what every person in the system has: name, identity, nationality.

***Why abstract?** You should never create a 'Person' directly - you always create a specific type like Pilot, Co-Pilot, or Passenger. Abstract classes cannot be instantiated.*

Fields:

Field	Access	Type	Explanation
name	private	String	Full name. Private for encapsulation - accessed via getter/setter.
nationality	private	String	Country of origin.

Methods: Standard getters and setters (getName, setName, etc.). Also a no-arg constructor Person().

OOP Concepts in this class:

- **Abstraction:** The class is abstract - it represents a concept, not a concrete thing
- **Encapsulation:** Fields are private, accessed through public getters/setters
- **Inheritance:** Employee and (should) Passengere extend this class
- **Constructor Overloading:** Two constructors: one with args, one no-arg

Interview Q: Why make Person abstract? Answer: To enforce that all subclasses share the same fields (name, identity, nationality) while preventing instantiation of an incomplete Person object.

8. Employee.java - Abstract Employee

Purpose: Intermediate abstract class between Person and concrete employee types (Flighter, modiefoun).

Fields: Only has **int ided** (static counter for ID generation). **Extends:** Person. **Extended by:** Flighter, modiefoun.

Why it exists: Provides a layer for employees (pilots and co-pilots) distinct from passengers. In a real system, employees would have salary, hire date, etc. Here it mostly serves as an architectural placeholder.

Warning: Passengere does NOT extend Person (design flaw). Passenger has duplicate name/identity/nationality fields instead of inheriting them.

9. Fighter.java - The Pilot Class

Purpose: Represents a pilot with salary, flight hours, and licenses (rokhas).

Field	Type	Explanation
id	int (instance)	Auto-incremented ID. Uses static ided counter.
hours	double	Total flight hours.
rokhasList	ArrayList<String>	List of licenses/certifications the pilot holds.

Salary Calculation: $\text{this.salary} = \text{salary} + (\text{hours} * \text{hourprice})$. This means a pilot with base salary 5000 and 100 hours gets: $5000 + (100 * 30) = 8000$.

10. modiefoun.java - The Co-Pilot Class

Purpose: Represents a co-pilot with salary, languages, and hours.

Fields: id, salary, langsList (ArrayList<String>), hours.

Extends: Employee. **Constructor:** Takes name, identity, nationality, salary, langsList, hours.

Note: The class name "modiefoun" appears to be a project-specific naming convention. In a professional codebase, this would be named "CoPilot".

Salary calculation differs from Fighter: setSalary calculates salary as $\text{hours} * \text{hourprice}$ (ignores the input salary). This is different from Fighter which adds base + hours.

11. Passengere.java - The Passenger Class

Purpose: Represents a passenger with passport details and destination.

Field	Type	Explanation
name, identity, nationality	String (package)	Package-private access. Should be private with inheritance from Person.
passEndDate	String	Passport expiry date.

Field	Type	Explanation
salary	double	Unused field. Passengers don't have salaries.

Design Flaw: Passengere does NOT extend Person despite having name, identity, nationality. It has duplicate fields. This violates DRY (Don't Repeat Yourself). It should extend Person.

12. status.java & Flight.java - Flight Model

status.java Purpose: Contains the FlightStatus enum with 4 values: SCHEDULED, BOARDING, DEPARTED, CANCELLED.

Flight.java Purpose: Represents a flight with destination, captain, passengers, co-pilots, seats, and status.

Key Design: Flight extends status (the class containing the enum). This is unusual - typically you'd compose an enum field, not inherit from the class containing it. But it works because Flight gets access to FlightStatus.

Field	Type	Explanation
id	int	Auto-incremented flight ID.
numberofchairs	int	Maximum passenger capacity.
passengersList	ArrayList<Passengere>	List of passengers on this flight.
flightStatus	FlightStatus	Current status from the enum.

13. checking.java - Input Validation

Purpose: Utility class for validating console input (Scanner).

Key Methods:

- **getValidInt()** - Reads an integer, returns 0 if invalid. Clears leftover newline.
- **getValidDouble()** - Reads a double, returns 0 if invalid.
- **entername()** - Prompts for and reads a name string.
- **enteridentity()** - Prompts for and reads an identity string.
- **enternationality()** - Prompts for and reads nationality.

Note: This class is used by adding.java for the console-based add methods. The GUI pages do NOT use this class - they use JavaFX TextField dialogs instead.

14. adding.java - Data Storage & Console Logic

Purpose: The central data storage class. Contains static ArrayLists that hold all system data.

Field	Type	Purpose
flightersList	static ArrayList<Flighter>	All pilots in the system
flightslists	static ArrayList<Flight>	All flights in the system

Console Methods (legacy, still in file):

- addFlighter() - Console-based pilot creation with Scanner
- addFlight() - Console-based flight creation (has infinite loop bug fixed)
- addcopilot() - Console-based co-pilot creation
- addairportattendants() - Console-based passenger creation

Critical Design: All data is in **static** fields. This means data is shared across ALL instances and persists for the JVM lifetime. However, it does NOT persist across application restarts - all data is lost.

15. Sidebar.java - Navigation Panel

Purpose: Left sidebar with logo, navigation buttons, and admin card.

Navigation Flow:

- User clicks a nav button
- Button's setOnAction calls mainApp.navigateTo(page)
- Main clears centerStack and creates the new page
- Sidebar tracks activeButton and activePage to highlight current nav
- If user clicks the already-active page, nothing happens (skip re-navigation)

Visual Design: Dark green background (#2F3A1D), white text, yellow accent (#CFFF74) for the active button. Three button styles: normal, hover, and active.

16. TopBar.java - Search & Status Bar

Purpose: Top bar with page title, search field, notification badge, date/time, and profile avatar.

Search System:

- TextField with Consumer<String> callback
- When text changes, callback fires -> Main.filterSearch(query)
- Main checks current page type and calls filterTable(query) on it
- Each page (FlightsPage, PassengersPage, etc.) implements filterTable()

Clock: Updates every second using a Timeline with INDEFINITE cycle count.

17. WelcomePage.java - Splash Screen

Purpose: Animated welcome screen shown before the main app.

Key Design Decisions:

- Extends StackPane (not VBox) for layering
- setMaxWidth(Double.MAX_VALUE) + setMaxHeight(Double.MAX_VALUE) to fill entire window
- Runnable onEnter callback: when button clicked, removes this overlay from baseRoot
- ScaleTransition on airplane: pulse animation (1.0 -> 1.06, 2.5s cycle)
- Decorative elements have setMouseTransparent(true) so they don't block clicks

Why it doesn't freeze: Unlike the old approach (primaryStage.hide/show), this uses a StackPane overlay. The main app loads behind it. No scene swapping needed.

18. DashboardPage.java - Overview Dashboard

Purpose: Main overview page with stats, management shortcuts, upcoming flights table, and quick actions.

Components:

- Welcome Banner: dark gradient with live counts
- 4 Stat Cards: colored cards with accent line on left
- Management Grid: 4 clickable cards linking to each section
- Upcoming Flights Table: shows all flights
- Quick Actions: shortcut buttons

Data Source: All stats come from adding.flightslists, adding.flightersList, adding.copilots (static ArrayLists).

19. FlightsPage.java - Flight Management

Theme: Blue theme

Key Details: TableView with status ComboBox (inline status change), add dialog with TextField for co-pilot count (allows 0+), delete button, filterTable() for search.

20. PassengersPage.java - Passenger Management

Theme: Green theme

Key Details: TableView loading passengers from all flights via loadAllPassengers(), add dialog, delete with removeFromFlights() helper, filterTable().

21. PilotsPage.java - Pilot Management

Theme: Orange theme

Key Details: TableView with SimpleStringProperty for salary formatting, licenses joined with String.join(), delete button, filterTable().

22. CoPilotsPage.java - Co-Pilot Management

Theme: Purple theme

Key Details: TableView with languages joined via `String.join()`, add dialog with comma-separated language input, `filterTable()`.

23. ReportsPage.java - Report Generation

Theme: Teal theme

Key Details: 5 report buttons, monospace TextArea, save-to-file via FileChooser + `Files.writeString()`.

24. ChatPage.java - LAN Chat

Theme: Red theme

Key Details: HBox layout (unlike other VBox pages), left sidebar for users, chat area with connect/send buttons, chatclient integration.

25. SettingsPage.java - System Info

Theme: Grey theme

Key Details: Display-only cards showing app info, Java version, data storage method.

26. chatclient.java - Socket Chat Client

Theme: N/A

Key Details: Connects to `server.py` via Socket, reads messages in background thread, updates TextArea via `Platform.runLater()`, UTF-8 encoding.

27-29. Legacy / Unused Files

- **Chatwindowcontroller.java** - Old chat window controller. Excluded from compilation. Was the original chat GUI before ChatPage replaced it.
- **App.java** - The original monolithic GUI (875 lines). All screens in one file. Replaced by the modular page system (Main + individual page files).
- **normalemployee.java** - Extends Employee. Has salary field. Not used anywhere in the current system. Was probably an earlier concept.

30. server.py - Python TCP Chat Server

Purpose: TCP socket server that handles real-time chat between multiple JavaFX clients on the same network (LAN).

Component	Explanation
<code>socket.socket(AF_INET, SOCK_STREAM)</code>	Creates a TCP socket (not UDP). <code>AF_INET</code> = IPv4, <code>SOCK_STREAM</code> = reliable ordered.
<code>bind(('0.0.0.0', 5555))</code>	Listens on all network interfaces on port 5555.

Component	Explanation
<code>listen()</code>	Puts socket in passive mode, waiting for connections.
<code>Thread(target=handle_client, daemon=True)</code>	Each client gets its own thread. <code>daemon=True</code> means threads die when main exits.
<code>client.recv(4096)</code>	Reads up to 4096 bytes from client. Returns empty string on disconnect.

Communication Flow:

- 1. Java client connects to server.py on port 5555
- 2. Client sends nickname (first message)
- 3. Server adds client to `active_clients` list
- 4. Server broadcasts 'nickname joined the chat!' to everyone
- 5. Client sends messages -> server broadcasts 'nickname: message' to ALL
- 6. When client disconnects, server broadcasts 'nickname left the chat!'
- 7. Server removes client from `active_clients` list

Why `\n` is critical: Java's `BufferedReader.readLine()` blocks until it receives a newline character. Without `\n` in `server.py`, messages would never appear on the client side.

PART III: TECHNICAL DEEP ANALYSIS

31. OOP Principles in This Project

Principle	Where Used	Example
Abstraction	Person.java, Employee.java	Person is abstract - you can't create a generic Person, only Flighter or Passengere
Inheritance	Flighter->Employee->Person, Flight->status	Flighter inherits name/identity/nationality from Person
Composition	Flight has Flighter, ArrayList<Passengere>	Flight 'has-a' captain (Flighter reference), 'has-a' passenger list
Static Members	adding.java, Colors.java	Static ArrayLists shared across all code. Static color constants.

Missing OOP: No interfaces are used. No abstract methods defined. No method overriding (except toString in some classes). No factory or builder patterns. The code could benefit from more polymorphism.

32. JavaFX GUI Analysis

Why JavaFX (not Swing)?

JavaFX is the modern successor to Swing. It supports CSS styling, FXML, better layouts, and richer components. However, this project uses NO FXML - all GUI is created programmatically in Java code.

Key JavaFX Components Used:

Component	What It Is	Used In
Stage	The window itself	Main.java (primaryStage)
BorderPane	Layout with 5 regions: top, bottom, left, right, center	Main.java - root layout

Component	What It Is	Used In
VBox	Vertical box layout	Every page extends VBox (or ChatPage extends HBox)
Button	Clickable button	Nav buttons, action buttons, save buttons
TextField	Single-line text input	Search bar, form fields, chat input
TableColumn	Column in TableView	Each field is a column with PropertyValueFactory or lambda
TextArea	Multi-line text area	Chat messages, report display
Circle	JavaFX shape (not Swing)	Icon backgrounds, avatar circles, accent dots

Layout Hierarchy:

```

Stage ── Scene (1600x900) ── StackPane (baseRoot) ── BorderPane (root) [main app, behind]
├─ Left: Sidebar (VBox, 250px)
│   └─ Logo (VBox)
│       └─ Nav buttons (Button[])
│           └─ Spacer (Region)
│               └─ Admin card (HBox)
├─ Top: TopBar (HBox, 70px)
│   └─ Title (Label)
│       └─ Search (TextField)
│           └─ Notification (StackPane)
│               └─ Date/Time (VBox)
│                   └─ Avatar (StackPane)
├─ Center: StackPane (centerStack)
│   └─ Current page (VBox/HBox)
│       └─ WelcomePage (StackPane) [overlay, on top]
│           └─ VBox layout
│               └─ Logo
│                   └─ Title
│                       └─ Features (HBox)
│                           └─ Airplane animation
│                               └─ Enter button

```

33. Socket Programming & Networking

TCP vs UDP: This project uses TCP (Transmission Control Protocol). TCP guarantees delivery, ordering, and error checking. UDP does not. Chat requires reliable delivery, so TCP is correct.

Socket Lifecycle:

Client Side (chatclient.java):

- 1. new Socket(serverIP, port) - initiates TCP handshake (SYN, SYN-ACK, ACK)
- 2. PrintWriter created with auto-flush for sending messages
- 3. BufferedReader created for reading incoming messages
- 4. out.println(nickname) - sends nickname to server
- 5. while ((message = in.readLine()) != null) - blocking read loop

- 6. `Platform.runLater()` -> `messageArea.appendText(message)` - updates JavaFX thread
- 7. On disconnect: `socket.close()`, `in.close()`, `out.close()` (finally block)

Why `Platform.runLater()`? JavaFX has a single UI thread. Only the UI thread can modify GUI components. The chat reader runs in a background thread (`Socket.read()` blocks). To update the `TextArea`, we must use `Platform.runLater()` to schedule the update on the UI thread.

34. Multithreading Analysis

Where threads are used:

- **chatclient.java:** `new Thread()` -> `connectToServer(serverIP, port).start()` - Background thread for socket I/O
- **server.py:** `threading.Thread(target=handle_client, daemon=True)` - Each client connection gets its own thread
- **JavaFX Timeline:** TopBar clock updates every second (runs on UI thread)
- **JavaFX FadeTransition:** Page transitions (250ms fade)
- **JavaFX ScaleTransition:** Welcome screen airplane pulse animation

Thread Safety: The chat system has a potential race condition: multiple threads access the `active_clients` list in `server.py`. This is handled with `threading.Lock()` (`clients_lock`).

35. File Handling

Where file handling is used:

- **ReportsPage.java:** `FileChooser` + `Files.writeString()` to save reports as .txt
- **chatclient.java:** Socket I/O (`InputStream/OutputStream`) - network file I/O
- **server.py:** Socket I/O + `print()` for console logging
- **NOT used:** No database, no property files, no serialization, no CSV reading

Data Persistence Issue: All data is in static `ArrayLists`. When the app restarts, ALL data is lost. A real system would use a database or file-based storage (e.g., `SQLite`, `JSON` files, or object serialization).

36. Exception Handling

Location	Exception	What Happens
Main.java <code>navigateTo()</code>	Exception (catch-all)	Shows error card instead of crashing. Prints stack trace.
PilotsPage <code>showAddPilotDialog()</code>	NumberFormatException	Shows error label for invalid salary/hours input.

Location	Exception	What Happens
chatclient.java	IOException	Connection error. Shows error in TextArea and console.
server.py	Exception (try/except)	Error from client. Server continues running.

37. Collections & Data Structures

Collection	Where Used	Why
<code>ArrayList<Flighter></code>	<code>adding.flightersList</code>	Dynamic size, fast index access, easy iteration
<code>ArrayList<Flight></code>	<code>adding.flightslists</code>	Same reasons as above
<code>ArrayList<String></code>	<code>Flighter.rokhasList</code> , <code>modiefoun.langsList</code>	Licenses and languages stored as strings

Time Complexity:

- Access by index: $O(1)$
- Search (contains): $O(n)$ - linear scan
- Add/remove at end: $O(1)$ amortized
- Add/remove at index: $O(n)$ - shifts elements

Alternative: For faster search, HashMap keyed by ID could be used. But since datasets are small, ArrayList is fine.

38. Design Patterns

Patterns Found:

- **Observer (partial):** JavaFX event listeners (`setOnAction`, `textProperty().addListener`) follow the Observer pattern - objects subscribe to change events.
- **Callback Pattern:** `Consumer<String>` in `TopBar` for search. `Runnable onEnter` in `WelcomePage`. These are callback functions.
- **Singleton-like:** `adding.java` uses static ArrayLists - effectively a singleton data store (only one instance of data exists, shared globally).
- **MVC-ish:** Data model (Flight, Flighter, etc.) are Model. GUI pages (FlightsPage, etc.) are View. `Main.java` acts as Controller (routes navigation, routes search).
- **Facade (partial):** `checking.java` provides a simplified interface for input validation, hiding the complexity of Scanner handling.

Missing Patterns (Improvements):

- **Factory:** Could use a PageFactory to create pages instead of switch-case in Main
- **Repository:** Data access should be in a separate Repository class, not static lists
- **Observer (full):** Pages could observe data changes and auto-refresh tables
- **Strategy:** Different search strategies (by name, by ID, by date) could be swappable

PART IV: EXAM & INTERVIEW PREPARATION

39. Execution Flow Timeline

```
Program starts (java Launcher) | v Launcher.main(args) | v Main.main(args) ->
launch(args) | v JavaFX Runtime initialized | v Main.start(Stage primaryStage) | ■■■■
Create baseRoot (StackPane) ■■■■ Create root (BorderPane) ■■■■ Create Sidebar ->
root.setLeft() ■■■■ Create TopBar -> root.setTop() ■■■■ Create centerStack
(StackPane) -> root.setCenter() ■■■■ navigateTo("dashboard") -> DashboardPage created
■■■■ Add root to baseRoot ■■■■ Create WelcomePage -> Add to baseRoot (on top) ■■■■
Scene(1600x900) -> primaryStage.show() | v User sees WelcomeScreen | v User clicks
"Enter System" | v welcomeHolder[0] removed from baseRoot | v User sees Dashboard |
■■■■ User clicks "Flights" in Sidebar ■ ■■■■ Main.navigateTo("flights") ->
FlightsPage ■■■■ User clicks "+ Add Flight" ■ ■■■■ FlightsPage.showAddFlightDialog()
-> Stage dialog ■ ■■■■ User fills form -> Flight created -> added to
adding.flightslists ■■■■ User types in Search Bar ■ ■■■■ TopBar callback ->
Main.filterSearch() -> FlightsPage.filterTable() ■■■■ User clicks "Chat" in Sidebar ■
■■■■ ChatPage created -> user enters nickname ■ ■■■■ Connect button -> chatclient
created -> Socket connects to server.py ■ ■■■■ User types message ->
chatclient.sendMessage() -> server broadcasts ■■■■ User clicks "Reports" ■■■■
ReportsPage -> User clicks "Flight Report" ■■■■ generateFlightReport() ->
StringBuilder -> TextArea ■■■■ User clicks "Save as .txt" -> FileChooser ->
Files.writeString()
```

40. Memory & JVM Analysis

Stack Memory: Local variables, method parameters, and method call frames. Each thread has its own stack.

Heap Memory: All objects (new keyword) live here. Shared across threads.

Method Area: Class definitions, static variables, constant pool.

Key Objects in Memory:

Object	Where Created	Lifetime
baseRoot (StackPane)	Main.start()	Entire app lifetime
Sidebar instance	Main.start()	Entire app lifetime
DashboardPage	navigateTo('dashboard')	Until next navigation (cleared)
Flight objects	FlightsPage dialog save	Until app restart (static list)

Object	Where Created	Lifetime
chatclient instance	ChatPage connectBtn	Until disconnect

Garbage Collection: When `centerStack.getChildren().clear()` is called during navigation, the old page's reference count drops to 0 and it becomes eligible for GC. The JVM's garbage collector frees the memory.

Static Memory: *adding.flightersList, adding.copilots, adding.flightslists are static - they live in the Method Area and are NEVER garbage collected until the JVM shuts down.*

41. Interview Questions (Per Class)

10 questions per class with answers. These cover common university and job interview topics.

Launcher.java

Q: Why do you need a Launcher class?

A: JDK 17's module system prevents launching classes that extend `Application` directly. `Launcher` is a plain class that bypasses this restriction.

Q: What happens if you remove `Launcher.java`?

A: You'd need to run `Main.main()` directly, which fails with a module system error. The fix would be adding `--add-opens` or using a different JVM flag.

Q: Is `Launcher` a design pattern?

A: It's a workaround/adaptor pattern - adapting the entry point to work around a system limitation.

Q: Could `Launcher` do more work?

A: Yes - it could initialize logging, load configuration, or set up security managers. Currently it just delegates to `Main`.

Q: What's the difference between `Launcher.main()` and `Main.main()`?

A: `Launcher.main()` is the JVM entry point. `Main.main()` calls `launch(args)` to start the JavaFX runtime and then calls `start(Stage)`.

Q: Why is `Launcher` not abstract?

A: It needs to be instantiated (or at least have its `main()` called). Abstract classes can't be instantiated, so it must be concrete.

Q: Could you use a static initializer block instead?

A: Yes, but `main()` is cleaner and more conventional. Static blocks run when the class loads, which could have ordering issues.

Q: What Java version requires this pattern?

A: JDK 17+ with strong encapsulation. JDK 8-11 didn't have this restriction.

Q: Is this pattern common in JavaFX projects?

A: Yes, it's the recommended approach in the official JavaFX documentation for modular projects.

Q: Could you use `module-info.java` instead?

A: Yes, a proper `module-info.java` with `requires javafx.controls` would also work, but the Launcher approach is simpler for small projects.

Main.java

Q: What is a `BorderPane` and why use it?

A: A layout with 5 regions (top, bottom, left, right, center). Perfect for dashboard apps: sidebar left, topbar top, content center.

Q: What is a `StackPane`?

A: A layout that stacks children on top of each other. Used for layering: main app behind, welcome screen on top.

Q: Why use `StackPane` instead of swapping scenes?

A: Swapping scenes causes flickering and doesn't work well with `primaryStage.show()`. `StackPane` overlay is smoother and more reliable.

Q: What is the `navigateTo()` method's try-catch for?

A: If any page creation fails (null pointer, missing data), the error is caught and an error card is shown instead of a blank screen.

Q: What is `FadeTransition`?

A: A JavaFX animation that smoothly transitions opacity from 0 to 1 over 250ms, creating a fade-in effect when changing pages.

Q: Why does `navigateTo()` clear `centerStack` first?

A: To remove the old page before adding the new one. `StackPane` holds only the current page, so clearing ensures no duplicate pages.

Q: What is the Consumer callback pattern?

A: `TopBar` registers a callback with `Main`. When search text changes, the callback fires, routing the search to the current page.

Q: Why track `currentPage` as a String?

A: To prevent re-navigation. If user clicks the already-active nav button, nothing happens (avoids unnecessary page recreation).

Q: What is the `welcomeHolder` array hack?

A: Java requires lambda-captured variables to be effectively final. Using an array allows modifying the contents (`welcomeHolder[0]`) while the array reference is final.

Q: How does `filterSearch` work?

A: It checks the type of the current node in `centerStack` using `instanceof`, then calls the appropriate `filterTable()` method on it.

Flighter.java / modiefoun.java (Employee Classes)

Q: Why does `Flighter` extend `Employee` instead of `Person` directly?

A: `Employee` adds a layer between `Person` and concrete employees. In a real system, employees would share attributes like hire date, department, etc.

Q: How are IDs generated?

A: Each class has a static int ided counter. When a new object is created, ided increments and is assigned to the instance's id field.

Q: What's wrong with this ID generation?

A: It's not thread-safe. If two threads create objects simultaneously, they might get the same ID. Also, IDs don't reset on restart.

Q: Why use ArrayList for licenses/languages?

A: Simple and flexible. Each license is just a string. No need for a License class unless you need to add attributes (expiry date, issuing authority).

Q: How is salary calculated in Flighter?

A: $\text{salary} = \text{inputSalary} + (\text{hours} * \text{hourprice})$. With $\text{hourprice} = 30$, a pilot with 100 hours gets $100 * 30 = 3000$ added to their base salary.

Q: Why is modiefoun's salary calculation different?

A: In `modiefoun.setSalary()`, $\text{salary} = \text{hours} * \text{hourprice}$ (ignores base). This might be a bug - it overwrites the input salary instead of adding to it.

Q: What is method overloading?

A: Having multiple methods with the same name but different parameters. Flighter has two constructors: one with all fields, one no-arg.

Q: What is the 'this' keyword doing in constructors?

A: It refers to the current object being created. `'this.salary = salary'` sets the instance field to the constructor parameter.

Q: Could Flighter and modiefoun be merged into one class?

A: Yes, with a role field (PILOT or COPILOT). But keeping them separate follows the Open/Closed principle - easy to add new roles without modifying existing code.

Q: What is the relationship between Flight and Flighter?

A: Composition - Flight 'has-a' Flighter (the captain). Flight holds a reference to a Flighter object, not a copy.

Passengere.java

Q: Why doesn't Passengere extend Person?

A: This is a design flaw. Passengere has duplicate name/identity/nationality fields. It should extend Person to avoid code duplication.

Q: What is DRY principle?

A: Don't Repeat Yourself. Passengere violates this by redefining fields that already exist in Person. The fix: class Passengere extends Person.

Q: Why is salary a field in Passengere?

A: It's unused and should be removed. Passengers don't have salaries. This was probably copied from Employee and never cleaned up.

Q: What is 'distination'?

A: A misspelling of 'destination'. In a professional codebase, this would be caught by code review and spell-checking tools.

Q: Why is 'name' package-private (no modifier)?

A: Accidental. It should be 'private' for proper encapsulation. Package-private means any class in the same package can access it directly.

Q: How does bookPassenger work?

A: Checks if `passengersList.size() < numberOfchairs`. If yes, adds passenger. If no, prints an error (in Arabic). Returns boolean success.

Q: Why does the error message print in Arabic?

A: The project appears to have been developed by Arabic-speaking developers. Some error messages are in Arabic while most UI is in English.

Q: How are passengers stored?

A: Each Flight has its own ArrayList. Passengers belong to flights, not stored globally. This is a relationship model.

Q: What is the difference between Passengere and PassengersPage?

A: Passengere is the data model (the object). PassengersPage is the GUI that displays and manages Passengere objects.

Q: Could you add a frequent flyer number field?

A: Yes - add a String field with getter/setter. The architecture supports it. You'd also need to update the PassengersPage table and add dialog.

Flight.java & status.java

Q: Why does Flight extend status?

A: This is unconventional. status contains the FlightStatus enum. By extending it, Flight gains access to the enum. Better approach: composition (have a FlightStatus field).

Q: What is an enum?

A: A type-safe class with a fixed set of constants. FlightStatus has exactly 4 values: SCHEDULED, BOARDING, DEPARTED, CANCELLED. Can't create new values at runtime.

Q: Why use enum instead of String for status?

A: Enums are type-safe. You can't have misspellings like 'SCHEDULD'. With Strings, any typo creates a new value. Enums also support switch statements.

Q: What is the relationship between Flight and Fighter?

A: Flight has a captain (Fighter) - composition. Flight also has a list of co-pilots (ArrayList) and passengers (ArrayList).

Q: How is the auto-incremented ID calculated?

A: `this.id = (this.id + (++this.ided))`. First increments the static counter, then adds to the instance's id. This creates sequential IDs.

Q: What happens if two Flights are created simultaneously?

A: The static ided counter could have a race condition. Two threads might get the same ID value. This is a thread-safety issue.

Q: Could Flight be immutable?

A: Partially. The status can change (`setFlightStatus`), so it's mutable. But destination, captain, and seats could be final for immutability.

Q: What is the FlightStatus enum's default values?

A: Every new flight starts as SCHEDULED. This is set in the constructor: `this.flightStatus = FlightStatus.SCHEDULED`.

Q: How many flights can exist at once?

A: Unlimited (bounded only by memory). They're stored in `adding.flightslists` ArrayList which grows dynamically.

Q: What is the Flight's toString() method used for?

A: Returns a formatted string with flight details. Useful for debugging and logging. Called automatically when printing a Flight object.

adding.java**Q: Why use static fields for data storage?**

A: For simplicity - all code can access `adding.flightersList` directly without passing references. In a real system, you'd use a database or repository pattern.

Q: What are the disadvantages of static storage?

A: No persistence (data lost on restart), no thread safety, no separation of concerns, hard to test, tight coupling between all classes and `adding`.

Q: How would you improve data storage?

A: Use a Database class with CRUD methods, or Repository pattern, or at minimum serialize to JSON/file for persistence.

Q: What is the relationship between adding and the GUI?

A: The GUI pages read/write directly to `adding`'s static lists. There's no abstraction layer - the GUI is tightly coupled to the data storage.

Q: Why are the console methods (addFlighter, etc.) still in adding.java?

A: They're legacy code from before the GUI was built. They should be removed or moved to a separate utility class.

Q: What is the 'choosencopilots' variable?

A: A static ArrayList used in the console `addFlight()` method to hold selected co-pilots. It had a bug where it was recreated inside a for loop.

Q: How does the Flight constructor handle null copiloList?

A: `if (copiloList == null) copiloList = new ArrayList<>()`. This prevents NullPointerException when creating a flight with no co-pilots.

Q: What is the time complexity of searching adding.flightslists?

A: $O(n)$ - linear scan. For each search, it iterates through all flights. For small datasets, this is fine. For large datasets, use HashMap.

Q: Could you use enums instead of strings for some fields?

A: Yes - destination could be an enum of cities, nationality could be an enum. But enums are fixed-size, so they limit flexibility.

Q: What happens if adding.flightslists is modified during iteration?

A: ConcurrentModificationException. If you remove a flight while iterating through the list, it crashes. This is why delete buttons use `remove()` separately.

Chat System (chatclient.java + server.py)

Q: What protocol does the chat use?

A: TCP (Transmission Control Protocol). Reliable, ordered delivery. Port 5555. Server is Python, client is Java.

Q: Why TCP instead of UDP?

A: Chat requires reliable delivery. UDP can drop packets, reorder messages, or lose data. TCP guarantees all messages arrive in order.

Q: What is a socket?

A: An endpoint for communication. Created with (IP address, port number). Client socket connects to server socket. Data flows through both.

Q: What is the role of threads in chat?

A: Server: each client gets a thread (handle_client). Client: reader runs in background thread so UI doesn't freeze. Main thread handles GUI.

Q: Why is Platform.runLater() needed?

A: JavaFX single-thread rule: only the UI thread can modify GUI components. Socket reader is in a background thread. Must schedule GUI updates on UI thread.

Q: What happens if server.py crashes?

A: Client's in.readLine() returns null. The while loop exits. chatclient sets isConnected = false. Client shows disconnection message.

Q: How are multiple clients handled?

A: Server uses threading. Each client connection gets a new Thread. Thread-safe access to active_clients via threading.Lock().

Q: What is the broadcast function?

A: Sends a message to ALL connected clients. Iterates through active_clients list and sends to each. Handles disconnects gracefully.

Q: Why append '\n' to messages?

A: Java's BufferedReader.readLine() reads until it sees a newline character. Without '\n', readLine() would block forever waiting for more data.

Q: What is SO_REUSEADDR?

A: A socket option that allows the server to bind to a port immediately after restarting, even if the previous connection is in TIME_WAIT state.

GUI Pages (FlightsPage, PassengersPage, etc.)

Q: Why does each page extend VBox?

A: VBox provides vertical layout (top-to-bottom). Each page is a vertical stack of components. ChatPage extends HBox for horizontal layout.

Q: What is PropertyValueFactory?

A: A JavaFX cell value factory that uses reflection to call getXxx() on the data object. PropertyValueFactory('name') calls getName() on the Flighter.

Q: What is SimpleStringProperty?

A: A JavaFX observable property that wraps a String. Used in lambda cell value factories to provide formatted values (e.g., salary with \$ prefix).

Q: How does `filterTable()` work?

A: Clears the table, iterates through all items, checks if any field contains the search query (case-insensitive), and adds matching items back to the table.

Q: What is `Modality.APPLICATION_MODAL`?

A: A dialog setting that blocks input to all other windows until the dialog is closed. Used for add/edit dialogs to prevent user interaction with the main window.

Q: How does the delete button work in `TableView`?

A: Each row has a custom `TableCell` with a `Button`. On click, it gets the item at that row's index, removes it from both the static list and the table.

Q: Why are color families (`LIGHT`, `MID`, `SOLID`) used?

A: For consistent theming. Each page type has a color family. `LIGHT` for backgrounds, `MID` for borders, `SOLID` for accents. Changes one file to update all pages.

Q: What is the dropshadow effect?

A: A CSS-based shadow effect applied to cards: `dropshadow(three-pass-box, rgba, 10, 0, 0, 3)`. Creates depth and visual hierarchy.

Q: How do buttons change color on hover?

A: `setOnMouseEntered` and `setOnMouseExited` event handlers change the button's CSS style string to a different background color.

Q: Why use `getStyle()` string replacement for hover?

A: JavaFX doesn't have a built-in hover class like CSS `:hover`. The code manually replaces color strings in the style. A better approach would be CSS pseudo-classes.

42. University Defense Questions (100+)

These are questions your professor might ask during a project defense. Answer each honestly and thoroughly.

Q1: Why did you choose OOP?

A: Because the project models real-world entities (people, flights) that share common attributes. OOP provides encapsulation, inheritance, and polymorphism to model these relationships cleanly.

Q2: Why JavaFX instead of Swing?

A: JavaFX is the modern replacement for Swing. It supports CSS styling, better layouts (BorderPane, StackPane), and is actively maintained. Swing is deprecated.

Q3: Why sockets for chat instead of a library?

A: Sockets demonstrate understanding of network programming fundamentals. Using a library like Socket.io would hide the networking details we needed to learn.

Q4: Why static fields in adding.java?

A: For simplicity in a small project. Static fields are globally accessible without passing references. In a larger system, we'd use dependency injection.

Q5: Why no database?

A: The project scope focused on OOP and GUI concepts, not persistence. Adding a database would require JDBC, SQL, and more complexity. In-memory storage demonstrates the concepts.

Q6: How does encapsulation work in your project?

A: Fields are private (e.g., private String name), accessed through public getters/setters. This protects data from direct modification and allows validation.

Q7: What is inheritance and where do you use it?

A: Inheritance is when a class extends another, inheriting its fields and methods. Flighter extends Employee extends Person. This avoids code duplication.

Q8: What is polymorphism?

A: Objects of different types treated as the same type. All pages extend VBox, so Main stores them in the same StackPane. Different behavior, same interface.

Q9: How does the search work?

A: TopBar has a TextField with a listener. When text changes, it calls Main.filterSearch(). Main checks the current page type and calls filterTable() on it.

Q10: Why is FlightStatus an enum?

A: Enums provide type-safe constants. You can't have invalid values like 'SCHEDULD'. Enums also support switch statements and are more readable than Strings.

Q11: How do you handle concurrent access in chat?

A: Server uses threading.Lock() to protect the active_clients list. Client uses Platform.runLater() to safely update the JavaFX UI from the socket reader thread.

Q12: What would you improve?

A: Add database persistence, implement proper MVC with repositories, add user authentication, use CSS for styling instead of inline styles, add input validation.

Q13: How does the welcome screen work?

A: It's a StackPane overlay on top of the main app. When the user clicks Enter, the overlay is removed from the base StackPane, revealing the main app underneath.

Q14: Why do you need Launcher.java?

A: JDK 17's module system prevents launching classes that extend Application directly. Launcher bypasses this by calling Main.main() from a non-Application class.

Q15: What design patterns do you use?

A: Callback pattern (Consumer for search), Observer (JavaFX event listeners), Singleton-like (static data), MVC-ish (Model: Flight, View: FlightsPage, Controller: Main).

Q16: How are IDs generated?

A: Each class has a static int counter (ided). When a new object is created, the counter increments and is assigned to the instance. IDs are sequential.

Q17: Why ArrayList instead of LinkedList?

A: ArrayList provides $O(1)$ random access by index, which is needed for TableView. LinkedList has $O(1)$ insert/remove but $O(n)$ access. For small datasets, both work.

Q18: What is the time complexity of your search?

A: $O(n)$ - linear scan through all items. For small datasets (<1000 items), this is acceptable. For larger datasets, HashMap would provide $O(1)$ lookup.

Q19: How does JavaFX event handling work?

A: Events (clicks, key presses) are dispatched to the source component. Components have setOnAction, setOnMouseClicked handlers that execute when the event occurs.

Q20: What happens if the chat server crashes?

A: Client's readLine() returns null, the while loop exits, isConnected becomes false, and an error message is shown in the TextArea.

Q21: Why use try-catch in navigateTo()?

A: If any page creation fails (e.g., null reference), the error is caught and an error card is shown instead of a blank screen. Prevents the app from crashing.

Q22: What is composition vs inheritance?

A: Composition: Flight HAS-A Fighter (captain). Inheritance: Fighter IS-A Employee. Composition is more flexible - you can change the captain without changing Fighter.

Q23: Why is Person abstract?

A: To prevent instantiation of an incomplete Person. You always create a specific type (Fighter, Passengere). Abstract classes define a contract for subclasses.

Q24: What is the Observer pattern?

A: Objects subscribe to events. When the event occurs, subscribers are notified. JavaFX uses this: buttons notify listeners when clicked, text fields notify when text changes.

Q25: How does the project handle errors?

A: try-catch blocks around risky operations. GUI shows error labels/dialogs. Server logs errors and continues. Main's navigateTo() catches all exceptions and shows error cards.

Q26: What is method overriding?

A: A subclass provides a specific implementation of a method defined in its parent. *Passengere* overrides *toString()* to provide a custom string representation.

Q27: Why use `String.format` for salary display?

A: `String.format("%.2f", salary)` formats the number with exactly 2 decimal places and a dollar sign. Makes the display consistent and professional.

Q28: What is the purpose of the no-arg constructor?

A: Allows creating objects without providing initial values. Used when you want to set fields later via setters. Required if a subclass calls `super()` with no args.

Q29: How does the `TableView` update?

A: When data changes (add/delete), `table.getItems().setAll()` or `.add().remove()` is called. `TableView` observes the `ObservableList` and redraws automatically.

Q30: What is an `ObservableList`?

A: A JavaFX list that notifies `TableView` when items are added/removed/changed. `TableView` auto-updates the display. Regular `ArrayList` doesn't provide this.

Q31: Why is the server in Python?

A: To demonstrate cross-language networking. Java client communicates with Python server via TCP sockets. Shows that networking is language-agnostic.

Q32: What is a daemon thread?

A: A thread that doesn't prevent the JVM from shutting down. Server uses `daemon=True` so threads die when the main program exits, avoiding resource leaks.

Q33: How do you ensure thread safety?

A: `Lock (server.py)` protects shared data. `Platform.runLater()` (client) ensures GUI updates happen on the JavaFX thread. Static fields are accessed sequentially in single-user mode.

Q34: What is the difference between `==` and `.equals()`?

A: `==` compares references (same object in memory). `.equals()` compares content. For Strings, always use `.equals()`. For enum comparison, `==` works.

Q35: Why use `StringBuilder` for reports?

A: String concatenation in loops creates many temporary String objects. `StringBuilder` modifies a buffer in place, much more efficient for building large strings.

Q36: What is autoboxing?

A: Automatic conversion between primitive types and their wrapper classes. `int -> Integer`, `double -> Double`. Used when adding primitives to generics (`ArrayList`).

Q37: What is the diamond operator `<>`?

A: Java 7+ feature that infers the generic type. `new ArrayList<>()` instead of `new ArrayList()`. Reduces code verbosity.

Q38: Why use `Platform.runLater()` instead of `Thread.sleep()`?

A: `runLater()` schedules on the UI thread. `Thread.sleep()` just pauses the current thread. `sleep()` on UI thread would freeze the entire app.

Q39: What is a callback?

A: A function passed as an argument to be executed later. Consumer in TopBar is a callback. When search text changes, Main's filterSearch is called.

Q40: How does the project handle null values?

A: Null checks before accessing methods: `if (f.getPassengersList() != null)`. Also null-safe string operations: `name != null ? name.toLowerCase() : ""`

Q41: What is StringJoiner/String.join()?

A: `String.join(', ', list)` joins all elements of a list with a separator. Used to display licenses and languages as comma-separated strings in tables.

Q42: Why wrap TableView in a card (VBox)?

A: For visual consistency. The card provides background, border-radius, shadow, and padding. This creates a cohesive design language across all pages.

Q43: What is CSS in JavaFX?

A: JavaFX supports CSS for styling. `-fx-background-color`, `-fx-border-radius`, `-fx-font-size` are CSS properties. Used extensively for inline styling in this project.

Q44: How does the project handle different screen sizes?

A: The app is resizable (`primaryStage.setResizable(true)`). Uses layout managers (`BorderPane`, `VBox`, `HBox`) that auto-resize. `setMaxWidth(Double.MAX_VALUE)` for full-width elements.

Q45: What is the purpose of Region spacer?

A: An empty Region used with `Priority.ALWAYS` in layouts. It pushes other elements to the edges, creating spacing. Common pattern in JavaFX layouts.

Q46: Why use Circle shapes for icons?

A: Instead of images or icons, the project uses Circle shapes with Label letters inside. This avoids image dependencies and works consistently across platforms.

Q47: What is the difference between VBox and StackPane?

A: `VBox` arranges children vertically (no overlap). `StackPane` stacks children on top (overlap). `StackPane` is used for layering (welcome over main).

Q48: How does the project handle input validation?

A: Number parsing with try-catch (`NumberFormatException`). Empty field checks with `.isEmpty()`. Server-side validation would be needed for production.

Q49: What is a Flowable in reportlab?

A: A renderable element that can be placed in a document. Custom Flowable (`ColBox`) creates colored info boxes. Wraps text and draws background rectangles.

Q50: What is the benefit of a centralized color system?

A: One file (`Colors.java`) defines all colors. Changing the sidebar color requires editing ONE line instead of 20+ files. Makes theming consistent and maintainable.

Q51: What is the MVC pattern?

A: Model: data objects (`Flight`, `Flighter`). View: GUI pages (`FlightsPage`). Controller: navigation logic (`Main`). This separation makes code more maintainable.

43. Suggested Improvements

Performance:

- Use `HashMap<Integer, Flight>` instead of `ArrayList` for $O(1)$ lookup by ID
- Implement lazy loading for large datasets
- Use background threads for data loading (`Platform.runLater` for UI updates)
- Cache computed values instead of recalculating on each access

Security:

- Add user authentication (login screen)
- Validate and sanitize all user input
- Encrypt chat messages (TLS/SSL)
- Use prepared statements if adding a database
- Don't store passwords in plain text (use hashing)

Architecture:

- Implement Repository pattern for data access
- Add proper MVC separation (Controller classes)
- Use dependency injection instead of static fields
- Create interfaces for data access (`IRepository`)
- Add service layer between GUI and data

GUI:

- Use external CSS file instead of inline styles
- Implement dark mode toggle
- Add loading spinners for async operations
- Use FXML for complex layouts (separate design from logic)
- Add keyboard shortcuts (Ctrl+S to save, etc.)

Data Persistence:

- Add SQLite database with JDBC
- Implement JSON file serialization
- Use JavaFX Properties for automatic UI updates
- Add data export/import functionality

Code Quality:

- Fix naming: `modiefoun` -> `CoPilot`, `Passengere` -> `Passenger`, `checking` -> `InputValidator`
- Add Javadoc comments to all public methods
- Fix the `Passengere` extends `Person` design flaw
- Remove unused files (`App.java`, `normalemployee.java`, `Chatwindowcontroller.java`)
- Add unit tests (JUnit 5)

- Fix the Flight constructor to accept null copiloList gracefully

44. ASCII Diagrams

Class Hierarchy Diagram:

```
Person (abstract) name: String identity: String nationality: String getName(),
getIdentity(), getNationality() setName(), setIdentity(), setNationality() | +--->
Employee (abstract) | ided: static int | | | +---> Flighter (Pilot) | | id: int | |
salary: double | | hours: double | | hourprice: double = 30 | | rokhasList: ArrayList
| | getSalary(): salary + (hours * hourprice) | | | +---> modiefoun (Co-Pilot) | id:
int | salary: double | langsList: ArrayList | hours: double | +---> (should extend
but doesn't) Passengere (standalone) id: int name, identity, nationality (DUPLICATE!)
passport: String passEndDate: String distination: String bookPassenger(): boolean
status (contains enum) FlightStatus: SCHEDULED | BOARDING | DEPARTED | CANCELLED |
+---> Flight (extends status) id: int destination: String numberOfchairs: int
nameofCaptin: Flighter (composition) passengersList: ArrayList copiloList: ArrayList
flightStatus: FlightStatus
```

Object Interaction Diagram:

```
Main (Controller) | |--- Sidebar (Navigation) | | | +--- Button[] (Nav buttons) | | |
| | +--- OnClick -> mainApp.navigateTo(page) | | | +--- Admin Card (HBox) | |---
TopBar (Status) | | | +--- TextField (Search) -> Consumer callback | +--- Label
(Title) -> setTitle() | +--- Timeline (Clock) | |--- StackPane (centerStack) | | |
+--- DashboardPage (VBox) | | +--- Stat Cards (4x StackPane) | | +--- Management Grid
(GridPane) | | +--- Flights Table (TableView) | | | +--- FlightsPage (VBox) | | +---
TableView | | +--- ComboBox (Status) | | +--- Dialog (Stage) | | | +--- ChatPage
(HBox) | +--- chatclient (Socket) | +--- TextArea (Messages) | +--- TextField (Input)
| |--- adding (Static Data) +--- flightslists: ArrayList +--- flightersList:
ArrayList +--- copilots: ArrayList +--- (Flight objects contain) +--- Passengere list
+--- Flighter reference +--- modiefoun list
```

Network Communication Diagram:

```
Java Client (chatclient.java) Python Server (server.py) =====
===== new Socket("localhost", 5555) -----> server_socket.accept()
| | +--- PrintWriter (output) -----> +--- client.recv(4096) | out.println(nickname)
| decode('utf-8') | out.println(message) | | +--- broadcast(msg, sender) +---
BufferedReader (input) <----- | for client in active_clients: | in.readLine() |
client.send(msg + "\n") | Platform.runLater( | | messageArea.appendText(msg)) +---
threading.Thread( | | target=handle_client, +--- isConnected = false | daemon=True)
(on disconnect) | ) | Message Flow: Message Flow: 1. Client connects 1. Server
accepts connection 2. Client sends nickname 2. Server adds to active_clients 3.
Client sends "Hello" 3. Server broadcasts to ALL 4. Server broadcasts 4. All clients
receive message 5. Client displays message 5. Client's readLine() returns msg 6.
Platform.runLater() updates UI
```

45. Complete Study Guide & Cheat Sheet

Java Keywords Used in This Project:

Keyword	Meaning	Where Used
public	Accessible from anywhere	All classes and main() methods

Keyword	Meaning	Where Used
static	Belongs to class, not instance	adding.flightersList, Colors.BACKGROUND, main()
class	Defines a blueprint	All 27 Java files
new	Creates object on heap	new Button(), new ArrayList<>(), new Flight()
void	Method returns nothing	main(), start(), navigateTo()
if/else	Conditional execution	Validation, null checks, navigation
try/catch	Exception handling	NumberFormatException, IOException
final	Cannot be changed after assignment	STYLE constants in GUI classes
instanceof	Checks object type at runtime	Main.filterSearch() checks page type

Cheat Sheet - Key Methods:

Class	Method	Purpose
Main	navigateTo(String)	Switch center page with fade animation
Sidebar	createNavButton()	Create styled button with hover/active states
FlightsPage	filterTable(String)	Filter flights by destination/captain/status/ID
adding	flightlists.add()	Add flight to static data store
ReportsPage	generateXxxReport()	Build report string with StringBuilder

Class	Method	Purpose
WelcomePage	onEnter.run()	Remove welcome overlay from baseRoot

46. Glossary of Java Keywords

abstract: A modifier for classes and methods that cannot be instantiated/overridden directly. Forces subclasses to provide implementation.

boolean: Primitive data type. True or false. Used for flags and conditions.

break: Exits a loop or switch statement prematurely.

catch: Block that handles an exception thrown in the try block.

class: A blueprint for creating objects. Defines fields and methods.

continue: Skips the current iteration of a loop and proceeds to the next.

double: Primitive. 64-bit floating point. Used for salary, hours, coordinates.

else: Alternative branch in an if statement.

enum: A special class that represents a group of constants (fixed values).

extends: Indicates that a class inherits from another class.

final: Prevents modification - final variable can't be reassigned, final method can't be overridden, final class can't be extended.

finally: Block that always executes after try/catch, regardless of exception.

for: Loop that repeats a block a specified number of times.

if: Conditional statement that executes code based on a boolean expression.

implements: A class implements an interface, providing concrete implementations of abstract methods.

import: Makes classes from other packages available in the current file.

instanceof: Tests whether an object is an instance of a specific class or subclass.

int: Primitive. 32-bit signed integer. Used for IDs, counts, indices.

new: Creates a new object on the heap and calls its constructor.

null: A special literal meaning 'no object'. Reference points to nothing.

package: Groups related classes. This project uses the default package (no package declaration).

private: Access modifier. Only accessible within the same class.

protected: Access modifier. Accessible within the same package and subclasses.

public: Access modifier. Accessible from anywhere.

return: Exits a method and optionally returns a value.

static: Belongs to the class itself, not to any instance. Shared across all objects.

super: Refers to the parent class. Used to call parent constructor or access parent methods.

switch: Selects one of many code blocks to execute based on a value.

this: Reference to the current object. Disambiguates field names from parameter names.

throw: Explicitly throws an exception.

throws: Declares that a method might throw a specific exception type.

try: Block of code that might throw an exception. Monitored for exceptions.

void: Indicates a method returns nothing.

while: Loop that repeats as long as a condition is true.

47. 50+ Question Quiz with Answers

Test your knowledge. Answers are provided after each question.

1. What is the entry point of this application?

A: Launcher.main()

2. Why can't Main.main() be the direct entry point?

A: JDK 17 module system blocks launching classes that extend Application

3. What layout does Main use as the root?

A: StackPane (baseRoot) containing a BorderPane (root)

4. What is the root layout's structure?

A: BorderPane: Sidebar (left), TopBar (top), centerStack (center)

5. What happens when you click a sidebar button?

A: Calls mainApp.navigateTo(page), which clears centerStack and creates the new page

6. Why use StackPane instead of Scene swapping?

A: Avoids flickering, no primaryStage.hide()/show() issues, smoother transitions

7. What is the WelcomePage overlay pattern?

A: WelcomePage sits on top of main app in baseRoot StackPane. Button removes it.

8. Why use WelcomePage[] welcomeHolder?

A: Java requires lambda-captured variables to be effectively final. Array trick allows delayed assignment.

9. What is Person?

A: Abstract base class with name, identity, nationality fields

10. Why is Person abstract?

A: Prevents instantiation. Forces creation of specific types (Flighter, Passengere)

11. What does Flighter extend?

A: Employee (which extends Person)

12. How is pilot salary calculated?

A: salary = inputSalary + (hours * hourprice) where hourprice = 30

13. What is modiefoun?

A: The Co-Pilot class, extends Employee

14. What fields does Passengere have?

A: name, identity, nationality, passport, passEndDate, distination

15. Why doesn't Passengere extend Person?

A: Design flaw - it has duplicate fields that should be inherited

16. What is status.java?

A: Contains FlightStatus enum: SCHEDULED, BOARDING, DEPARTED, CANCELLED

17. What does Flight extend?

A: status (the class containing the FlightStatus enum)

18. What are Flight's key fields?

A: id, destination, numberofchairs, nameofCaptin (Flighter), passengersList, copiloList, flightStatus

19. How does Flight handle null copiloList?

A: if (copiloList == null) copiloList = new ArrayList<>()

20. What is adding.java?

A: Central data storage with static ArrayLists for all system data

21. What are the three static lists in adding?

A: flightersList (pilots), copilots (co-pilots), flightslists (flights)

22. What is checking.java?

A: Input validation utility for console-based Scanner input

23. What does Colors.java do?

A: Defines all color constants used across the UI

24. How many color families are there?

A: 7: Flights (blue), Passengers (green), Pilots (orange), Co-Pilots (purple), Reports (teal), Chat (red), Settings (grey)

25. What does each color family have?

A: LIGHT (background), MID (border), SOLID (accent) variants

26. How does the search bar work?

A: TopBar has Consumer callback -> Main.filterSearch() -> current page's filterTable()

27. How does filterTable() work in FlightsPage?

A: Clears table, iterates all flights, checks if any field contains query (case-insensitive), adds matches

28. What is SimpleStringProperty?

A: JavaFX observable wrapper for Strings, used in lambda cell value factories

29. Why not use StringBinding?

A: StringBinding is abstract, can't be instantiated as anonymous class easily. SimpleStringProperty is simpler.

30. How does the status ComboBox work?

A: TableColumn with custom CellFactory that shows a ComboBox. Selecting a value updates FlightStatus.

31. What is ChatPage's layout?

A: HBox (not VBox): left sidebar (users) + right chat area

32. What is chatclient.java?

A: Socket client that connects to server.py, reads messages in background thread

33. How does chatclient update the GUI?

A: Platform.runLater() schedules TextArea update on the JavaFX UI thread

34. What is server.py?

A: Python TCP socket server on port 5555, handles multiple clients with threading

35. Why does server append \n to messages?

A: Java's `BufferedReader.readLine()` blocks without newline. Server must send `\n` for `readLine()` to return.

36. What is `threading.Lock()`?

A: Mutex that protects shared data (`active_clients`) from concurrent access

37. How does `ReportsPage` save files?

A: `FileChooser` shows save dialog, `Files.writeString()` writes content to chosen file

38. What are the 5 report types?

A: Flight, Passenger, Pilot, Co-Pilot, Full Summary

39. How is the welcome airplane animated?

A: `ScaleTransition` with `INDEFINITE` cycle, pulsing from 1.0 to 1.06 scale

40. What is `Modality.APPLICATION_MODAL`?

A: Dialog setting that blocks all other windows until dialog is closed

41. How do buttons change color on hover?

A: `setOnMouseEntered/setOnMouseExited` handlers change the CSS style string

42. What is a `PropertyValueFactory`?

A: JavaFX utility that uses reflection to call `getXxx()` on data objects for table columns

43. What is the advantage of `ArrayList` over `LinkedList`?

A: $O(1)$ random access by index, better for `TableView` which accesses items by position

44. What is the time complexity of search?

A: $O(n)$ - linear scan through all items

45. Why use `StringBuilder` for reports?

A: More efficient than String concatenation in loops (avoids creating many temporary String objects)

46. What is the difference between public and private?

A: Public: accessible from anywhere. Private: accessible only within the same class.

47. What is method overloading?

A: Multiple methods with same name but different parameters (e.g., `Flighter` has two constructors)

48. What is method overriding?

A: Subclass provides specific implementation of parent's method (e.g., `Passengere` overrides `toString()`)

49. What is composition?

A: Object contains references to other objects (`Flight HAS-A Flighter captain`)

50. What is the difference between `==` and `.equals()`?

A: `==` compares references, `.equals()` compares content

51. What is autoboxing?

A: Automatic conversion between primitives and wrapper classes (`int <-> Integer`)

52. What is the diamond operator `<>`?

A: Java 7+ infers generic type: `new ArrayList<>()` instead of `new ArrayList()`

53. What is the Observer pattern?

A: Objects subscribe to events. When event occurs, subscribers are notified.

54. What is the Callback pattern?

A: Function passed as argument, executed later. Consumer in TopBar.

55. What is the Singleton pattern?

A: Only one instance exists. Static fields in `adding.java` create a global data store.

56. What is the MVC pattern?

A: Model (data), View (GUI), Controller (navigation). `Main.java` acts as controller.

End of Document

Airport Management System - Complete Educational Guide